



테스트메이트

TestMate

말하면, 테스트 데이터가 만들어집니다.

채팅으로 원하는 상황을 말하면, AI가 필요한 API를 골라 실제로 실행해 테스트 데이터를 만들어 주는 도구입니다.

참가분야

상태

내부 업무 AI 활용 (AX)

동작하는 프로토타입 · 데모 서버 6종 검증

## 1 어떤 문제를 푸는가

“결제까지 마친 회원 데이터, 또 만들어야 해.”

QA와 개발자는 테스트할 때마다 이런 “상황별 데이터”를 직접 만들어야 합니다. 채용 서비스라면 — 서류합격 상태 지원자, 면접대기 지원자, 유료 광고를 결제한 기업회원처럼 **상황마다 다른 데이터**가 필요합니다.

이 데이터를 만들려면 어떤 기능을 **어떤 순서로** 거쳐야 하는지 알아야 하고(회원가입 → 이력서 작성 → 공고 지원 → 기업계정 전환 → 서류합격 처리...), 경우의 수가 많아 **미리 다 만들 수 없으며**, 매번 여러 화면을 오가며 **손으로 반복**해야 합니다. 화면이 아직 없는 개발 초기엔 개발자에게 요청하거나 도구로 직접 API를 호출해야 해 시간이 더 듭니다.

**핵심 문제** — 정작 테스트는 시작도 안 했는데, “준비”라는 부수 작업에만 시간이 반복해서 녹습니다.

## 2 무엇을 만들었나

사용자는 방법을 몰라도 됩니다. **원하는 상황만 말하면** AI가 등록된 작업(레시피)들 중 필요한 것을 골라 순서대로 실행합니다. 처음 보는 요청이라도 그 자리에서 필요한 API를 조합합니다.

**비결은 “실제 API로만 만든다”는 것** — 데이터베이스를 직접 건드리지 않고, 실제 서비스가 쓰는 **API로만** 데이터를 만듭니다.

예를 들어 “이력서 열람권을 3건 쓴 기업회원”을 DB에 직접 넣으면 결제·차감 로직을 건너뛰어 **현실에 없는 상태**가 됩니다. API로 만들면 실제 사용자와 똑같은 절차(검증·부수효과·정합성)를 거치므로 **현실과 똑같은 데이터가 안전하게** 만들어집니다.

주문하고 결제까지 한번에 처리해줘

주문 후 결제하기

주문을 생성하고 결제한다.

필요한 값

상품 ID \*

수량 \*

결제수단 \*

1

1

CARD

기본값

기본값

기본값

바로 실행

값 확인 후 실행

실행됨

주문 후 결제하기 완료

1. 주문 생성 — POST /orders → 201

2. 결제 — POST /payments → 201

결제 완료

주문번호: 1002

결제번호: 5001

결제금액: 19,900원

결제상태: PAID

▶ 상세 값 보기

결과 보기

실제 화면 — 상황을 말하면 AI가 레시피를 실행하고 결과를 정리해 보여준다

**동작하는 프로토타입입니다.** 서로 다른 도메인의 데모 서버 6종에 실제로 연결해 실행되며, 뒤의 정량 수치도 실제 실행 로그에 근거합니다.

## 3 대표 시나리오 (채용 서비스 예시)

“○○ 공고에 지원해서 서류합격 상태인 지원자 만들어줘”

AI가 두 종류의 계정·여러 화면을 넘나들며 순서대로 실행합니다.

1

구직자

회원이입 또는 로그인

2

구직자

메일 인증번호를 스크립트로 읽어 인증

3

구직자

운영 중인 공고 중 하나 탐색

4

구직자

해당 공고에 지원

5

기업

방금 들어온 지원 건을 서류합격으로 변경

**결과** — 지원자 계정 · 지원한 공고 · 서류합격 상태까지 한 번에 준비됩니다. 사람이 하면 계정 두 개를 오가며 여러 화면을 거쳐야 하는 일을 **한 문장**으로 끝냅니다.

## 4 AI를 어떻게 활용했나

### ① 제품 안에서 — AI가 핵심 동력

- 자연어 요청을 이해해 필요한 API를 **직접 고르고 순서를 조합**합니다.
- 부족한 정보는 실제 API 목록과 사내 문서(Confluence)를 **스스로 찾아** 보완합니다.
- 이 “매번 다른 요청을 이해하고 조합하는” 판단은 **규칙 기반 자동화로는 불가능**하고 AI만 할 수 있습니다. AI가 장식이 아니라 핵심 동력인 이유입니다.

### ② 개발 과정에서 — AI를 “팀처럼” 활용

1인 개발이지만 AI를 단순 코드 자동완성이 아니라 **협업 팀처럼** 활용했습니다.

- 기획 우선** — 기능마다 먼저 기획 문서를 AI와 함께 구조화하고, 확정 후 구현에 착수하며 문서와 코드를 계속 동기화했습니다.
- 규칙의 내재화** — 코딩 컨벤션·보안·커밋·에러 처리 규칙을 AI가 항상 참조하는 규칙(steering)으로 정해, 매번 같은 기준 위에서 작업하도록 했습니다.
- 역할 분리** — 기획·백엔드·프론트엔드·리뷰를 전문 역할(서비스에이전트)로 나눠 병렬 진행하고 교차 검토했습니다.

이 방식으로 기획·백엔드·프론트엔드·문서를 1인이 진행해 **동작하는 프로토타입**을 완성했습니다.

## 5 차별점 — “그거 그냥 ○○로 되는 거 아냐?”

| 기존 방식                   | 한계                            | 테스트메이트                                 |
|-------------------------|-------------------------------|----------------------------------------|
| DB에 직접 INSERT           | 실제 절차를 건너뛰어 비현실·깨진 데이터, 위험    | <b>실제 API로만 → 현실과 동일</b>               |
| Swagger / Postman 수동 호출 | 순서·값을 사람이, API를 알아야 함         | <b>말 한마디로 AI가 순서 조합</b>                |
| 미리 짠 자동화 스크립트           | 정해진 경우만, 바뀌면 다시 코딩            | <b>처음 보는 요청도 즉석 조합</b>                 |
| 범용 AI 코딩 도구             | 코드를 만들어 줄 뿐, 사내 API 연결·실행은 각자 | <b>사내 API 연결 + 실제 실행 + 재사용 자산으로 즉석</b> |

이 도구만의 3가지

- 실제 API로만 (DB 안 건드림)** — 안전하고 현실과 일치하는 데이터.
- 즉석 조합 (AI만 가능)** — 규칙 기반 자동화로는 불가능한 지점.
- 불이면 자동 등록·확산** — 서버에 라이브러리만 추가하면 API 목록이 자동 등록됩니다. 상태 서버의 DB 구조를 몰라도 되고, 서버가 재기동되면 최신 API로 자동으로 맞춰집니다. **연결된 서비스가 늘수록 더 강해집니다.**

## 6 정량 성과 (추정 + 실측)

### A) 테스트 데이터 준비 시간 — 추정

가정(보수적 예시): 채용 시나리오 “서류합격 지원자 1건” 수동 준비 ≈ **8분**(두 계정·여러 화면 포함), QA가 스프린트마다 **상태 조합 5종**을 반복.

| 구분       | 수동    | 테스트메이트       |
|----------|-------|--------------|
| 1건 준비    | 약 8분  | <b>약 30초</b> |
| 상태 5종 1회 | 약 40분 | <b>약 3분</b>  |

약 90%

준비 시간 절감

40분 → 3분

상태 5종 준비 (1회)

※ 실측이 아닌 시나리오 기반 추정이며 가정 값을 명시했습니다. 단계가 많은 시나리오일수록 절감폭은 더 커집니다.

### B) AI 호출 비용 — 실측 (OpenRouter 로그 기반)

- 의도 해석·작업 선택 1회 = 입력 **약 1,150~2,330 토큰** / 출력 **약 10~50 토큰**, 호출당 **약 \$0.003(약 4원)**.
- 한 작업(보통 1~3회 호출) = **대략 4~12원** 수준.
- 출력 토큰이 입력의 1/60 수준 → 자연어 장문이 아니라 “작업 선택”만 받으므로 생성 비용이 최소화됩니다.

## 7 AI 자원 효율성 (토큰 절감 설계)

같은 기능도 순진하게 만들면 토큰을 몇 배씩 씁니다. 테스트메이트는 비용을 처음부터 설계에 반영했습니다.

| 설계          | 효과                                                            |
|-------------|---------------------------------------------------------------|
| 작업 선택 1회 호출 | 의도 분석과 작업 선택을 한 번에 (2단계 → 1회) → 호출 수 절반                       |
| 레시피 조건부 전달  | 서비스 미지정 땀 레시피를 안 보냄, 지정 땀 해당 서비스만(15~20개) → 실측 입력 토큰 최대 2배 차이 |
| 모델 이원화      | 무거운 판단은 고성능 모델, 가벼운 처리는 저가 모델로 분리                             |
| 레시피 직접 실행   | 자주 쓰는 작업은 AI를 거치지 않고 버튼으로 바로 실행 → AI 호출 0회                    |
| 조회 루프 상한    | 정보 조회는 최대 5회·120초로 제한 → 폭주·과금 방지                              |

**직접 실행의 절감 효과 (실측 단가 기반)** — 자주 쓰는 작업은 처음 한 번만 AI로 레시피화하면 이후엔 버튼으로 바로 실행돼 AI 호출이 필요 없습니다. 예: 같은 작업을 10회 수행할 때 매번 AI로 하면 10회 호출, 레시피 직접 실행을 쓰면 **2회로 감소(80%↓)**. 호출당 약 4원이므로 40원 → 8원.

## 8 신뢰성 · 안전장치 그리고 한계

### 안전장치

- 지어내지 않습니다.** 실제 등록된 API 목록과 문서를 근거로만 답하고 실행합니다. 근거가 없으면 되물거나 “없다”고 답합니다.
- 위험한 작업은 확인받습니다.** 결제처럼 되돌릴 수 없는 API는 표시( @TestForgeConfirm )로 실행 전 확인을 거치고, 내부·관리용 API는 아예 노출되지 않게( @TestForgeExclude ) 막을 수 있습니다.
- 내 권한 그대로 실행.** API 호출이 사용자 브라우저에서 이뤄져, 이미 로그인한 내 세션·권한을 그대로 재사용합니다. 시스템이 남의 인증 정보를 대신 보관하지 않습니다.
- 운영 환경 영향 없음.** API 목록은 개발·스테이징 환경에서만 전송하도록 설정하며, 라이브러리를 설정 파일로 커야만 동작합니다.

### 한계 (프로토타입 — 정직하게)

- 아직 프로토타입 단계입니다. 데이터 되돌리기, 대량 생성, 레시피 공유 등은 다음 단계 과제입니다.
- 크로스 도메인 실행에는 외부 서버의 쿠키 설정 등 전제 조건이 필요합니다.

## 9 로드맵 · 확산

테스트메이트의 본질은 “**대화로 사내 API를 골라 실제로 실행하는 엔진**”입니다. 지금 대상은 “테스트 데이터 생성”이지만, **대상만 바꾸면 같은 골격을 다른 업무에 재사용**할 수 있습니다.

지금

QA 테스트 데이터 생성 (동작 프로토타입)

다음

사내 서비스로 확산. 라이브러리만 불이면 새 서버도 자동 연동되어 **확산 비용이 낮음**

이후

아래 두 방향으로 확장

① 대화형 통합 어드민

“OO기업 최근 결제 내역이랑 지금 올라간 공고 보여줘” → 조회 API들을 골라 실행 → 한 화면에 정리

② 대화형 사용자 서비스

“내가 지원한 공고 중 서류합격한 곳만 알려줘” → 지원현황 조회 API 실행 → 정리해 답변

**공통 구조** — 세 가지 모두 “대화 → AI가 알맞은 API 선택 → 실제 실행 → 결과 정리”라는 **같은 골격**을 씁니다. 그래서 확장은 새로 만드는 일이 아니라, 이미 만든 엔진의 적용 범위를 넓히는 일입니다.

**지금은 QA 데이터 생성에 집중하지만, 같은 구조가 사내 어드민·사용자 서비스까지 하나의 대화형 실행 플랫폼으로 자라날 수 있습니다.**

테스트메이트 (TestMate) · 프로젝트 설명 문서 · 내부 업무 AI 활용(AX)